SORTING & SEARCHING
ALGORITHM NOTES

OVERVIEW

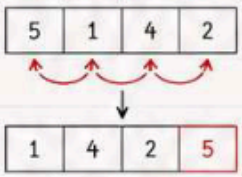
Sorting arranges data in a particular order (ascending / descending).
Searching finds the location or existence of an element in data.

SORTING ALGORITHMS

1. BUBBLE SORT

- Repeatedly swap adjacent elements if they are in wrong order.
- Time: $O(n^2)$ • Space: $O(1)$

Example (Pass 1):



2. SELECTION SORT

- Find the minimum element and place it in correct position.
- Time: $O(n^2)$ • Space: $O(1)$

Example:



3. INSERTION SORT

- Build sorted array one item at a time by inserting each element.
- Time: $O(n^2)$ • Space: $O(1)$

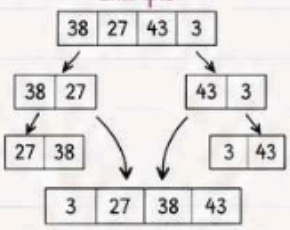
Example:



4. MERGE SORT

- Divide array into halves, sort them and merge the sorted halves.
- Time: $O(n \log n)$ • Space: $O(n)$

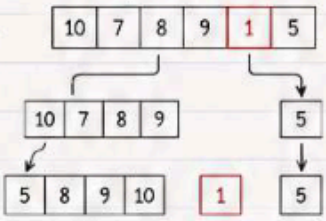
Example:



5. QUICK SORT

- Choose a pivot, partition the array and recursively sort subarrays.
- Avg Time: $O(n \log n)$
  Worst Time: $O(n^2)$
- Space: $O(\log n)$

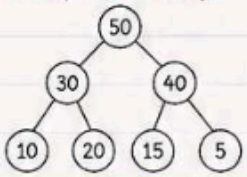
Example (Pivot = 1):



6. HEAP SORT

- Build a heap from array and repeatedly extract root (max/min) and heapify.
- Time: $O(n \log n)$ • Space: $O(1)$

Example (Max Heap):

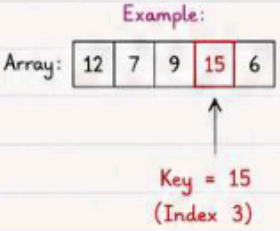


SEARCHING ALGORITHMS

1. LINEAR SEARCH

- Check each element one by one until the key is found or array ends.
- Time: $O(n)$ • Space: $O(1)$

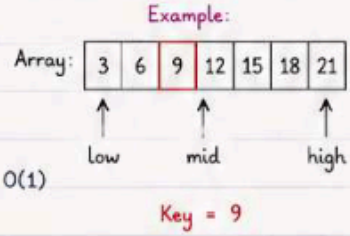
Example:



2. BINARY SEARCH

- Works on sorted array.
- Compare middle element and search in half.
- Time: $O(\log n)$ • Space: $O(1)$

Example:



SORTING vs SEARCHING

SORTING	SEARCHING
• Arranges data in a particular order.	• Finds position of an element.
• Useful for multiple operations.	• Used for retrieval operations.
• Time consuming comparatively.	• Faster (if data is sorted).

KEY TAKEAWAY :

- ✓ Choose the right algorithm based on data size, type and requirement.
- ✓ Sorting helps in organizing data for efficient processing.
- ✓ Searching helps in quick retrieval of information. ☆

☆ Learn it. Practice it. Master it. ☆

BUBBLE SORT

1. WHAT IS BUBBLE SORT?

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. This process is repeated until the list is sorted.

2. HOW IT WORKS?

- Compare the first two elements.
- If the first > second → swap them.
- Move to the next pair and repeat.
- After each pass, the largest element "bubbles" to the end.
- Repeat for remaining unsorted part until no swaps are needed.

3. ALGORITHM (PSEUDOCODE)

```
BubbleSort(arr)
  n ← length of arr
  for i ← 0 to n-2                  // number of passes
      swapped ← false
      for j ← 0 to n-i-2            // compare adjacent
          if arr[j] > arr[j+1]
              swap(arr[j], arr[j+1])
              swapped ← true
      if swapped = false            // already sorted
          break
```

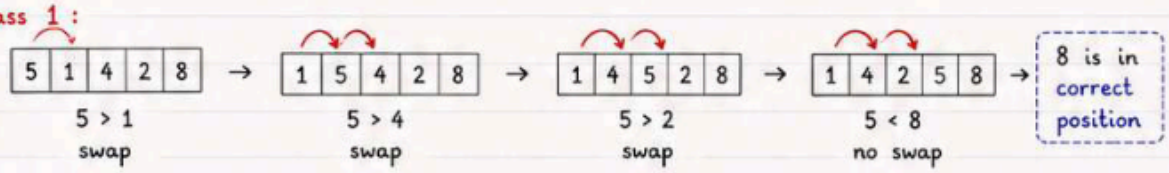
4. COMPLEXITY

- Best Case : O(n) (Already sorted)
- Average Case : $O(n^2)$
- Worst Case : $O(n^2)$ (Reverse sorted)
- Space Complexity : O(1) (In-place sorting)

5. EXAMPLE

Given array : | 5 | 1 | 4 | 2 | 8 |

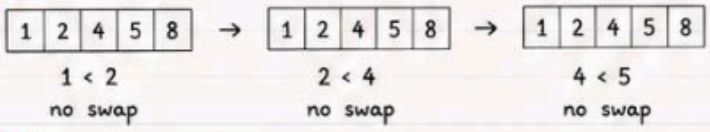
Pass 1 :



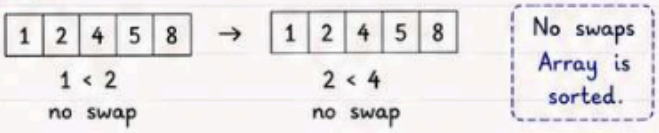
Pass 2 :



Pass 3 :



Pass 4 :



6. CHARACTERISTICS

- Simple to understand and implement.
- In-place sorting algorithm.
- Stable algorithm (does not change the relative order of equal elements).
- Not suitable for large datasets (higher time complexity).

Pro Tip: If no two elements are swapped in a pass, the array is already sorted. We can stop early to save time.

☆ Learn it. Practice it. Master it. ☆

SELECTION SORT

1. WHAT IS SELECTION SORT?

Selection Sort is a simple comparison-based sorting algorithm.
It divides the array into two parts:
a sorted part (at the beginning)
and an unsorted part (at the end).
In each pass, it selects the minimum element from the unsorted part and places it at the beginning of the unsorted part.

2. HOW IT WORKS?

- Find the minimum element in the unsorted part.
- Swap it with the first element of the unsorted part.
- Move the boundary of the unsorted part one step forward.
- Repeat until the array is sorted.

3. ALGORITHM (PSEUDOCODE)

```
SelectionSort(arr)
  n ← length of arr
  for i ← 0 to n-2              // last element will
      min_idx ← i               // assume first unsorted
      for j ← i+1 to n-1        // find minimum in
          if arr[j] < arr[min_idx]   // unsorted part
              min_idx ← j
      swap(arr[i], arr[min_idx])     // place minimum at
                                     // correct position
```

4. COMPLEXITY

- Best Case : $O(n^2)$ (Already sorted)
- Average Case : $O(n^2)$
- Worst Case : $O(n^2)$ (Reverse sorted)
- Space Complexity : $O(1)$ (In-place sorting)

5. EXAMPLE

Given array : | 64 | 25 | 12 | 22 | 11 |

Pass 1 :

64	25	12	22	11	→	11	25	12	22	64

Unsorted part

Minimum (11) is found and swapped with first element.

Pass 2 :

11	25	12	22	64	→	11	12	25	22	64

Unsorted part — Unsorted part

Minimum (12) in unsorted part swapped with first element of unsorted part.

Pass 3 :

11	12	25	22	64	→	11	12	22	25	64

Unsorted part

Minimum (22) in unsorted part swapped.

Pass 4 :

11	12	22	25	64	→	11	12	22	25	64

Unsorted part

Minimum (25) in unsorted part swapped.

Pass 5 :

11	12	22	25	64	→	11	12	22	25	64

Only one element left (64) already in place.

Array is sorted.

6. CHARACTERISTICS

- Simple and easy to understand.
- In-place sorting algorithm.
- Not stable (relative order of equal elements may change).
- Not efficient for large datasets ($O(n^2)$ time complexity).

Pro Tip : Selection Sort is good for small datasets or when memory writes are costly as it does minimum number of swaps.

☆ Learn it. Practice it. Master it. ☆

INSERTION SORT

1. WHAT IS INSERTION SORT?

Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time.

It takes each element from the unsorted part and inserts it into its correct position in the sorted part.

2. HOW IT WORKS?

- Start from the second element (index 1).
- Compare it with elements in the sorted part (left side).
- Shift greater elements one position to the right.
- Insert the element at its correct position.
- Repeat for all remaining elements.

3. ALGORITHM (PSEUDOCODE)

```
InsertionSort(arr)
  n ← length of arr
  for i ← 1 to n-1              // start from 2nd element
      key ← arr[i]              // element to insert
      j ← i-1                   // index of last element
      while j ≥ 0 and arr[j] > key   // shift greater
          arr[j+1] ← arr[j]          elements to right
          j ← j-1
      arr[j+1] ← key            // insert key at
                                   correct position
```

4. COMPLEXITY

- Best Case : $O(n)$ (Already sorted)
- Average Case : $O(n^2)$
- Worst Case : $O(n^2)$ (Reverse sorted)
- Space Complexity : $O(1)$ (In-place sorting)

5. EXAMPLE

Given array : | 12 | 11 | 13 | 5 | 6 |

Pass 1 (i = 1) :

key = 11

12	11	13	5	6	→	11	12	13	5	6

11 < 12, shift 12 right — insert 11 at position 0

Pass 2 (i = 2) :

key = 13

11	12	13	5	6	→	11	12	13	5	6

13 > 12, no shift — already in correct position

Pass 3 (i = 3) :

key = 5

11	12	13	5	6	→	5	11	12	13	6

5 < 13, shift 13 right
5 < 12, shift 12 right
5 < 11, shift 11 right
insert 5 at position 0

Pass 4 (i = 4) :

key = 6

5	11	12	13	6	→	5	6	11	12	13

6 < 13, shift 13 right
6 < 12, shift 12 right
6 < 11, shift 11 right
6 > 5, stop shifting
insert 6 at position 1

Array is sorted.

6. CHARACTERISTICS

- Simple and easy to understand and implement.
- In-place sorting algorithm.
- Stable algorithm (maintains relative order of equal elements).
- Efficient for small datasets or nearly sorted data.
- Not suitable for large datasets ($O(n^2)$ time complexity).

KEY IDEA

At every step, take one element from the unsorted part and insert it into its correct position in the sorted part.



Pro Tip : Insertion Sort is efficient for small or nearly sorted arrays and has very low overhead as it does not require extra space.

☆ Learn it. Practice it. Master it. ☆

MERGE SORT

1. WHAT IS MERGE SORT?

Merge Sort is a divide-and-conquer sorting algorithm.
It divides the array into halves, sorts each half recursively and then merges the sorted halves to get the final sorted array.

2. HOW IT WORKS?

- Divide the array into two halves.
- Recursively sort both halves.
- Merge the two sorted halves.
- Repeat until the entire array is sorted.

3. ALGORITHM (PSEUDOCODE)

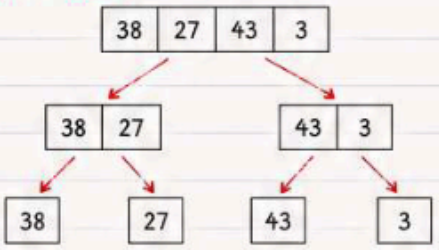
```
MergeSort(arr, l, r)
  if l < r                          // more than one element
     mid ← (l + r) // 2             // find middle
     MergeSort(arr, l, mid)         // sort left half
     MergeSort(arr, mid + 1, r)     // sort right half
     Merge(arr, l, mid, r)          // merge sorted halves
```

```
Merge(arr, l, mid, r)
Create two temp arrays:
   L ← arr[l...mid]     R ← arr[mid+1...r]
i ← 0, j ← 0, k ← l
while i < size(L) and j < size(R)
   if L[i] ≤ R[j]
      arr[k] ← L[i];  i ← i + 1
   else
      arr[k] ← R[j];  j ← j + 1
   k ← k + 1
Copy remaining elements of L (if any)
Copy remaining elements of R (if any)
```

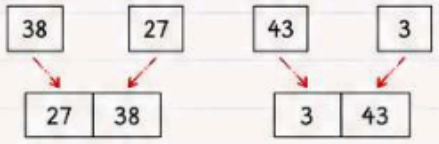
4. EXAMPLE

Given array : 38 | 27 | 43 | 3

Step 1 : Divide



Step 2 : Conquer (Sort individual parts)



Step 3 : Merge



5. MERGE PROCESS (TWO SORTED ARRAYS)

Example : L = 12 | 20 | 40 R = 15 | 18 | 30

12 20 40 (i)	15 18 30 (i, j)	Result → 12 (12 < 15)
12 20 40 (i)	15 18 30 (j)	Result → 12 15 (15 < 20)
12 20 40 (i)	15 18 30 (j)	Result → 12 15 18 (18 < 20)
12 20 40 (i)	15 18 30 (j)	Result → 12 15 18 20 (20 < 30)
12 20 40 (i)	15 18 30 (j)	Result → 12 15 18 20 30 (30 < 40)
12 20 40 (i)	15 18 30 (j)	Result → 12 15 18 20 30 40

(Remaining element 40 is copied)

6. COMPLEXITY

- Best Case : O(n log n)
- Average Case : O(n log n)
- Worst Case : O(n log n)
- Space Complexity : O(n) (Not in-place)

7. CHARACTERISTICS

- Stable sorting algorithm.
- Not in-place (requires extra space).
- Efficient for large datasets.
- Uses Divide and Conquer approach.

Pro Tip : Merge Sort is very efficient for large datasets and linked lists where random access is costly.

☆ Learn it. Practice it. Master it. ☆

QUICK SORT

1. WHAT IS QUICK SORT?

Quick Sort is a divide-and-conquer sorting algorithm.
It picks a "pivot" element, partitions the array into elements smaller and greater than the pivot, and recursively sorts the subarrays.

2. HOW IT WORKS?

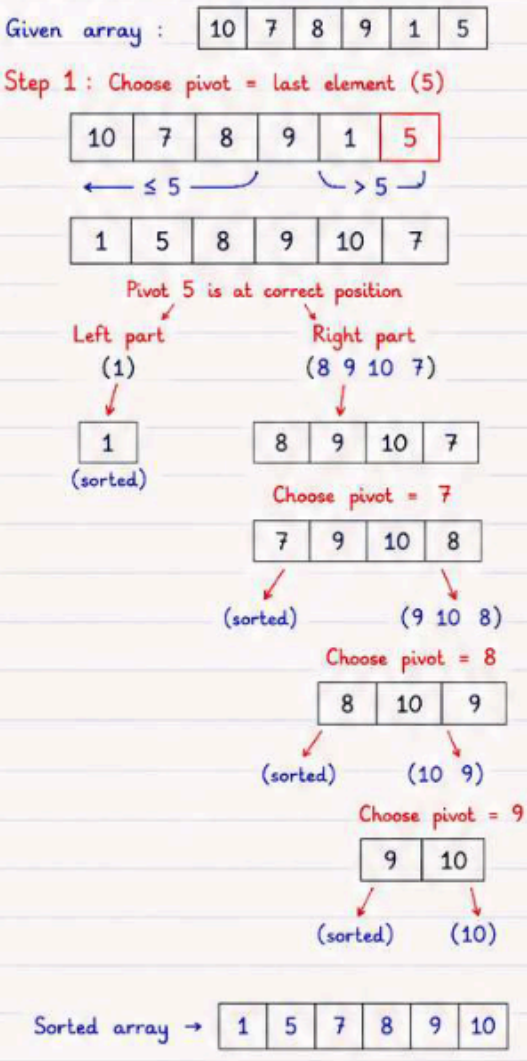
- Choose a pivot element.
- Partition the array so that elements less than pivot are on the left and greater than pivot are on the right.
- Recursively apply Quick Sort on left and right subarrays.

3. ALGORITHM (PSEUDOCODE)

```
QuickSort(arr, low, high)
  if low < high
     pi ← Partition(arr, low, high)      // pivot index
     QuickSort(arr, low, pi - 1)         // sort left part
     QuickSort(arr, pi + 1, high)        // sort right part
```

```
Partition(arr, low, high)
 pivot ← arr[high]                       // choose last element as pivot
 i ← low - 1                             // index of smaller element
 for j ← low to high - 1
    if arr[j] ≤ pivot
       i ← i + 1
       swap(arr[i], arr[j])              // move smaller element left
 swap(arr[i + 1], arr[high])             // place pivot in correct place
 return i + 1                            // return pivot index
```

4. EXAMPLE



5. COMPLEXITY

- Best Case : $O(n \log n)$ (Pivot divides array equally)
- Average Case : $O(n \log n)$
- Worst Case : $O(n^2)$ (Pivot is smallest or largest)
- Space Complexity : $O(\log n)$ (Due to recursion stack)
  In-place sorting

6. CHARACTERISTICS

- Efficient and widely used sorting algorithm.
- In-place sorting (requires extra space for recursion stack only).
- Not stable (relative order of equal elements may change).
- Performance depends on choice of pivot.
- Used in many real-world applications due to its speed.

PIVOT SELECTION STRATEGIES

- Last element (as used above)
- First element
- Random element
- Median of three (first, middle, last)

Pro Tip : Choosing a good pivot (like random or median of three) helps to avoid worst case and keeps Quick Sort efficient.

☆ Learn it. Practice it. Master it. ☆

HEAP SORT

1. WHAT IS HEAP SORT?

Heap Sort is a comparison-based sorting algorithm that uses a binary heap data structure.
It first builds a max-heap from the array, then repeatedly extracts the maximum element and places it at the end of the array.

2. HOW IT WORKS?

- Build a max-heap from the given array.
- The largest element (root) is swapped with the last element.
- Reduce the heap size by one.
- Heapify the root to restore max-heap property.
- Repeat until the heap size becomes 1.

3. ALGORITHM (PSEUDOCODE)

```
HeapSort(arr)
    n ← length of arr
    // Build max-heap
    for i ← [n/2] - 1 downto 0
        Heapify(arr, n, i)
    // Extract elements one by one
    for i ← n - 1 downto 1
    swap(arr[0], arr[i])    // Move current root to end
    Heapify(arr, i, 0)      // Restore heap property

Heapify(arr, n, i)
    largest ← i         // Initialize largest as root
    l ← 2*i + 1         // left child index
    r ← 2*i + 2         // right child index

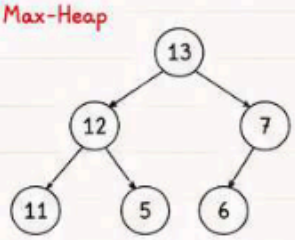
    if l < n and arr[l] > arr[largest]
        largest ← l
    if r < n and arr[r] > arr[largest]
        largest ← r

    if largest ≠ i
        swap(arr[i], arr[largest])
        Heapify(arr, n, largest)   // Recursively heapify
```

4. EXAMPLE

Given array : | 12 | 11 | 13 | 5 | 6 | 7 |

Step 1: Build Max-Heap



Max-Heap array : | 13 | 12 | 7 | 11 | 5 | 6 |

Step 2: Extract elements one by one

	Swap (root with last)		Heapify	Sorted Part
1.	(13 ↔ 6)	6 12 7 11 5 **13**	→ 12 11 7 6 5 **13**	13
2.	(12 ↔ 5)	5 11 7 6 12 **13**	→ 11 6 7 5 12 **13**	12 13
3.	(11 ↔ 5)	5 6 7 11 12 **13**	→ 7 6 5 11 12 **13**	11 12 13
4.	(7 ↔ 5)	5 6 7 11 12 **13**	→ 6 5 7 11 12 **13**	7 11 12 13
5.	(6 ↔ 5)	5 6 7 11 12 **13**	→ 5 6 7 11 12 **13**	5 6 7 11 12 13

Sorted array : | 5 | 6 | 7 | 11 | 12 | 13 |

5. COMPLEXITY

- Best Case : O(n log n)
- Average Case : O(n log n)
- Worst Case : O(n log n)
- Space Complexity : O(1) (In-place sorting)

6. CHARACTERISTICS

- In-place sorting algorithm.
- Not stable (relative order may change).
- Efficient for large datasets.
- Uses Binary Heap (Max-Heap).
- Always O(n log n) time complexity.

KEY IDEA

Use a Max-Heap so that the largest element is always at the root. Repeatedly move it to the end and rebuild the heap.

Pro Tip : Heap Sort is great when you need guaranteed O(n log n) time and in-place sorting with low extra space.

☆ Learn it. Practice it. Master it. ☆

LINEAR SEARCH

1. WHAT IS LINEAR SEARCH?

Linear Search is the simplest searching algorithm. It checks each element of the array one by one from the beginning until the key is found or the end of the array is reached.

It works on both sorted and unsorted arrays.

2. HOW IT WORKS?

- Start from the first element of the array.
- Compare the current element with the key.
- If equal, return the index (position).
- If not equal, move to the next element.
- Repeat until the key is found or array ends.
- If key is not found, return -1.

3. ALGORITHM (PSEUDOCODE)

```
LinearSearch(arr, n, key)
    for i ← 0 to n - 1
        if arr[i] == key
            return i      // key found at index i
    return -1             // key not found
```

where,

- arr → array
- n → number of elements in array
- key → element to be searched

4. EXAMPLE

Given array :

23	34	12	45	9	56	31

Key to search : 45

Step	i (index)	arr[i]	Comparison	Result
1	0	23	23 == 45 ?	No
2	1	34	34 == 45 ?	No
3	2	12	12 == 45 ?	No
4	3	45	45 == 45 ?	Yes
5	4	9	—	Key found at index 3
6	5	56	—	—
7	6	31	—	—

Output : Key 45 found at index 3.

Another Example (Key Not Found)

Array :

10	20	30	40	50

Key : 35

Checking : 10 ≠ 35 → 20 ≠ 35 → 30 ≠ 35 → 40 ≠ 35 → 50 ≠ 35 → End

Output : Key 35 not found. Return -1.

5. COMPLEXITY

- Best Case : O(1)
  (Key is the first element)
- Average Case : O(n/2) = O(n)
- Worst Case : O(n)
  (Key is the last element or not present)
- Space Complexity : O(1)
  (In-place search)

6. CHARACTERISTICS

- Very easy to understand and implement.
- Works on both sorted and unsorted arrays.
- Inefficient for large datasets due to O(n) time.
- Useful for small data or when data is not sorted.

KEY POINTS



- Checks elements sequentially.
- Stops as soon as key is found.
- Returns index if found, otherwise -1.
- No extra space required.

Pro Tip : Linear Search is good for small or unsorted data.
For large or sorted data, use Binary Search for better efficiency.

☆ Learn it. Practice it. Master it. ☆

BINARY SEARCH

1. WHAT IS BINARY SEARCH?

Binary Search is an efficient searching algorithm. It works on a sorted array. It repeatedly divides the search interval in half. If the key is equal to the middle element, it is found. If the key is smaller, search in the left half. If bigger, search in the right half. This process continues until the key is found or the interval becomes empty.

2. HOW IT WORKS?

- The array must be sorted.
- Find the middle element.
- Compare the key with the middle element.
- If equal, return the index.
- If key is smaller, search in the left half.
- If key is larger, search in the right half.
- Repeat until key is found or interval becomes empty.
- If key is not found, return -1.

3. ALGORITHM (PSEUDOCODE)

```
BinarySearch(arr, n, key)
    low ← 0                          // starting index
    high ← n - 1                     // ending index
    while low ≤ high
        mid ← (low + high) // 2      // middle index
        if arr[mid] == key
            return mid               // key found
        else if key < arr[mid]
            high ← mid - 1           // search left half
        else
            low ← mid + 1            // search right half
    return -1                        // key not found
```

4. EXAMPLE

Given sorted array :

5	12	19	27	33	45	58	63	71

Key to search : **33**

Step	low	high	mid	arr[mid]	Comparison	Result
1	0	8	4	33	33 == 33 ?	Yes
2	–	–	–	–	–	–

Output : Key 33 found at index 4.

Another Example (Key Not Found)

Array :

5	12	19	27	33	45	58	63	71

Key : **20**

Step	low	high	mid	arr[mid]	Comparison	Action
1	0	8	4	33	20 < 33	Search Left (high = 3)
2	0	3	1	12	20 > 12	Search Right (low = 2)
3	2	3	2	19	20 > 19	Search Right (low = 3)
4	3	3	3	27	20 < 27	Search Left (high = 2)
5	3	2	–	–	low > high	Not Found

Output : Key 20 not found. Return -1.

5. COMPLEXITY

- Best Case : O(1)
  (Key is the middle element)
- Average Case : O(log n)
- Worst Case : O(log n)
- Space Complexity : O(1)
  (In-place search)

6. CHARACTERISTICS

- Works only on sorted arrays.
- Very fast compared to Linear Search.
- Reduces the search space by half in each step.
- Not suitable for unsorted data.
- Widely used in real world applications.

KEY POINTS

- Always apply Binary Search on sorted data.
- Check middle element to decide the direction.
- Eliminate the half that cannot contain the key.
- Repeat until key is found or interval is empty.
- Efficient and reliable searching technique.

Pro Tip : Binary Search is much faster than Linear Search for large and sorted datasets. It reduces time complexity from O(n) to O(log n).

☆ Learn it. Practice it. Master it. ☆